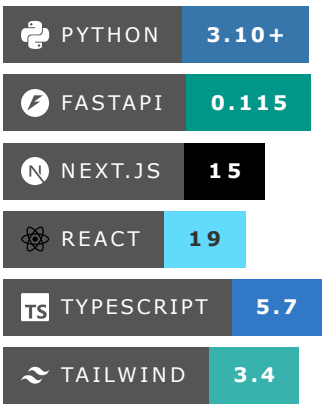
TSMOM — Quantitative Strategy Platform

Production-Grade Time-Series Momentum Engine with Full-Stack Analytics Dashboard



Based on the landmark research by Moskowitz, Ooi & Pedersen (2012) — "Time Series Momentum"

What This Project Does

This is a **complete quantitative trading research platform** that:

1. **Downloads** 20+ years of market data for 27 ETFs across 4 asset classes
2. **Generates** momentum signals using multi-horizon lookbacks (1m, 3m, 6m, 12m)
3. **Detects** market regimes (Trending vs Mean-Reverting) using Hidden Markov Models
4. **Constructs** portfolios with volatility-targeting and constraint management
5. **Backtests** strategies with realistic transaction costs, slippage, and drawdown protection
6. **Serves** results through a blazing-fast REST API with intelligent caching
7. **Displays** everything in a futuristic glassmorphism web dashboard
8. **Explains** results in plain English for non-technical investors

---

Screenshots

Strategy Dashboard

> Real-time KPI cards (CAGR, Sharpe, Sortino, Max DD, Calmar, Vol, Hit Rate), interactive equity curve with benchmark overlay and regime shading, drawdown visualization, rolling metrics, portfolio exposure, current positions, monthly returns heatmap, and historical stress tests.

```
┌──────────────────────────────────────────────────────────────────┐
│  TSMOM                                                           │
│  Quant Platform      ┌──────────────────────────────────────────┐│
│                      │ Strategy Dashboard                  LIVE │
│  ▸ Dashboard         │                                          │
│    Signals           │ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐     │
│    Regime            │ │ CAGR │ │SHARPE│ │MAX DD│ │ VOL  │     │
```

```
│    Risk              │ │-0.28%│ │-0.01 │ │-23.7%│ │6.26% │     │ │
│    Backtest Lab      │ └──────┘ └──────┘ └──────┘ └──────┘     │ │
│    Research          │                                          │ │
│    Equity Report     │ ╔═══════════════════════════════════╗ │ │
│    Live Trading      │ ║     📈 Equity Curve + Benchmark      ║ │ │
│                      │ ║     (Interactive Plotly Chart)       ║ │ │
│  ☀ Light Mode        │ ╚═══════════════════════════════════╝ │ │
│  v2.0.0              │                                          │ │
│                      │ ╔═══════════════════════════════════╗ │ │
│                      │ ║     📉 Drawdown | Rolling Metrics    ║ │ │
│                      │ ╚═══════════════════════════════════╝ │ │
│                      │                                          │ │
│                      │  Positions │ Monthly Heatmap │ Stress   │ │
│                      └──────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────┘
```

Equity Research Report (NEW)

> Plain-English investment analysis for non-technical investors. Shows portfolio grade, risk profile, market regime, per-asset momentum verdicts, and actionable recommendations.

```
┌──────────────────────────────────────────────────┐
│  Equity Research Report              AI-POWERED  │
│                                                  │
│  Portfolio: [Core] [Growth] [Income] [Commodities]│
│                                                  │
│  ╔═══════════════════════════════════════════╗   │
│  ║  EXECUTIVE SUMMARY                        ║ │
│  ║  Portfolio returned 0.7% per year with a  ║ │
│  ║  Sharpe ratio of 0.18                     ║ │
│  ║                                           ║ │
│  ║  Grade: FAIR │ Risk: MODERATE │ Trending ✓ ║ │
│  ╚═══════════════════════════════════════════╝   │
│                                                  │
│  ┌──────────────────────────────────────────┐   │
│  │ ☑ GLD — Favorable    │ Annual: +11.8%     │ │
│  │   Strong upward momentum                  │ │
│  ├──────────────────────────────────────────┤   │
│  │ ⚠  SPY — Neutral     │ Annual: +9.2%      │ │
│  │   No clear direction                      │ │
│  ├──────────────────────────────────────────┤   │
│  │ ✕ TLT — Unfavorable  │ Annual: -0.5%      │ │
│  │   Strong downward momentum                │ │
│  └──────────────────────────────────────────┘   │
└──────────────────────────────────────────────────┘
```

Additional Pages

Page	Description
Signals	Signal heatmap grid, per-asset signal decomposition, pipeline visualization
Regime	HMM regime score history, probability chart, trending vs mean-reverting stats
Risk	VaR/CVaR at 95% & 99%, rolling risk, drawdown analysis, risk contribution pie, stress tests
Backtest Lab	Interactive parameter controls (vol target, lookbacks, drawdown), asset toggles, presets, CSV export
Research	Academic foundation with KaTeX equations, glossary, references to MOP (2012) paper
Live Trading	Real-time simulation with 5-second auto-refresh, animated positions, signal gauge chart

Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          TSMOM Platform                              │
├──────────────────────────┬───────────────────────────────────────────┤
│     Frontend (Next.js)   │         Backend (FastAPI)                 │
│                          │                                           │
│  React 19 + TypeScript   │   REST API (/api/*)                       │
│  Plotly.js Charts        │   ┌──────────────────────────────┐        │
│  TanStack React Query ───┼──→│ In-Memory Cache (10min TTL)  │        │
│  Framer Motion           │   └──────────────────────────────┘        │
│  Glassmorphism UI        │              │                            │
│  Tailwind CSS            │   ┌──────────▼──────────────────┐        │
│                          │   │     Service Layer            │        │
│  Pages:                  │   │  ┌────────────────────────┐ │        │
│  /dashboard              │   │  │   Quant Engine (src/)  │ │        │
│  /dashboard/signals      │   │  │   ├─ Backtest Engine   │ │        │
│  /dashboard/regime       │   │  │   ├─ Signal Generator  │ │        │
│  /dashboard/risk         │   │  │   ├─ Regime Detector   │ │        │
│  /backtest               │   │  │   ├─ Portfolio Builder │ │        │
│  /research               │   │  │   ├─ Risk Calculator   │ │        │
│  /equity-research  ★NEW  │   │  │   └─ Cost Model        │ │        │
│  /live                   │   │  └────────────────────────┘ │        │
│                          │   └──────────────────────────────┘        │
├──────────────────────────┴───────────────────────────────────────────┤
│                          Data Layer                                  │
│  Yahoo Finance API → Parquet Cache (data/cache/) → 27 ETFs           │
│  20+ years of daily OHLCV data (2005–present)                        │
└──────────────────────────────────────────────────────────────────────┘
```

Quick Start

Prerequisites

- **Python 3.10+** (with pip)
- **Node.js 18+** (with npm)
- **Git**

1. Clone & Install

```
# Clone the repository
git clone https://github.com/YOUR_USERNAME/time-series-momentum.git
cd time-series-momentum

# Install Python dependencies
pip install -r backend/requirements.txt

# Install frontend dependencies
cd frontend && npm install && cd ..
```

2. Start the Backend (Terminal 1)

```
python start_backend.py
```

Wait for: `Application startup complete.`
Backend runs at: **http://localhost:8000**

3. Start the Frontend (Terminal 2)

```
cd frontend
npm run dev
```

Frontend runs at: **http://localhost:3000**

4. Open Your Browser

Navigate to **http://localhost:3000** — the dashboard loads automatically.

> **First load takes ~10 seconds** (computing 20 years of backtests). All subsequent loads are **< 0.5 seconds** thanks to intelligent caching.

API Endpoints

All endpoints are prefixed with `/api/`.

Method	Endpoint	Description	Response Time
POST	/run-backtest	Full backtest with custom parameters	~10s cold / ~0.4s cached
GET	/get-signals	Momentum signals for specified assets	~8s cold / ~0.3s cached
GET	/get-regime	HMM-based market regime detection	~6s cold / ~0.3s cached
GET	/get-risk	VaR, CVaR, drawdown, stress tests	~10s cold / ~0.4s cached
GET	/live-update	Simulated live trading snapshot	~5s cold / ~0.3s cached
GET	/research-report	Plain-English equity research report	~12s cold / ~0.3s cached
GET	/get-performance	Quick performance summary	~10s cold / ~0.4s cached
GET	/universe	Available asset universes & presets	Instant
GET	/health	Health check	Instant

Example API Calls

```
# Health check
curl http://localhost:8000/api/health

# Run a backtest
curl -X POST http://localhost:8000/api/run-backtest \
  -H "Content-Type: application/json" \
  -d '{"tickers":["SPY","QQQ","TLT","GLD","IEF"],"start_date":"2005-01-01"}'

# Get signals
curl "http://localhost:8000/api/get-signals?
tickers=SPY,QQQ,GLD&start_date=2010-01-01"

# Get equity research report
curl "http://localhost:8000/api/research-report?tickers=SPY,QQQ,TLT,GLD,IEF"

# Live trading simulation
curl "http://localhost:8000/api/live-update?tickers=SPY,QQQ,TLT,GLD"
```

Project Structure

```
time-series-momentum/
│
├── backend/                          # FastAPI backend
│   ├── main.py                       # App entry point, CORS, router mounting
│   ├── requirements.txt              # Python dependencies
│   ├── app/
│   │   ├── api/
│   │   │   └── routes.py             # 9 REST endpoints
```

```
│   │   ├── core/
│   │   │   ├── config.py            # Pydantic settings (env-configurable)
│   │   │   └── database.py          # SQLAlchemy async engine
│   │   ├── models/
│   │   │   └── schemas.py           # 20+ Pydantic request/response models
│   │   └── services/
│   │       └── engine.py            # Service layer + in-memory cache
│   └── Dockerfile
│
├── frontend/                        # Next.js 15 frontend
│   ├── app/
│   │   ├── layout.tsx               # Root layout (sidebar, providers, mesh
bg)
│   │   ├── page.tsx                 # Redirect → /dashboard
│   │   ├── globals.css              # Glassmorphism, glow effects, animations
│   │   ├── dashboard/
│   │   │   ├── page.tsx             # Main dashboard (KPIs, charts,
positions)
│   │   │   ├── signals/page.tsx     # Signal analysis
│   │   │   ├── regime/page.tsx      # Regime detection
│   │   │   └── risk/page.tsx        # Risk analytics
│   │   ├── backtest/page.tsx        # Interactive backtest lab
│   │   ├── research/page.tsx        # Academic research (KaTeX equations)
│   │   ├── equity-research/page.tsx # Plain-English investor reports ★
│   │   └── live/page.tsx            # Live trading simulation
│   ├── components/
│   │   ├── charts/                  # Plotly chart wrappers
│   │   ├── layout/                  # Sidebar, Header
│   │   └── ui/                      # Card, Button, Badge, KPI, Loading
│   ├── lib/
│   │   ├── api.ts                   # Type-safe API client
│   │   ├── providers.tsx            # React Query + Theme providers
│   │   └── utils.ts                 # Formatters, cn()
│   ├── .env.local                   # API URL config
│   ├── package.json
│   ├── tailwind.config.ts
│   └── tsconfig.json
│
├── src/                             # Quant engine (core Python library)
│   ├── data/
│   │   ├── provider.py              # YFinance + Parquet cache layer
│   │   ├── cleaning.py              # Returns, winsorization, alignment
│   │   └── universe.py              # YAML universe loader
│   ├── signals/
│   │   ├── momentum.py              # TSMOM signal, multi-horizon blend
│   │   ├── composite.py             # Regime-aware composite signals
│   │   ├── regime.py                # HMM regime detector
│   │   ├── filters.py               # Smoothing, z-score, capping
│   │   └── volatility.py            # EWMA, Yang-Zhang, Garman-Klass
│   ├── portfolio/
│   │   ├── construction.py          # Portfolio constructor
│   │   ├── position_sizing.py       # Volatility-targeted weights
│   │   └── constraints.py           # Position & exposure constraints
```

```
│   ├── risk/
│   │   ├── metrics.py              # 30+ KPIs (Sharpe, Sortino, etc.)
│   │   ├── drawdown.py             # Drawdown control overlay
│   │   ├── var.py                  # Parametric, historical, conditional VaR
│   │   └── stress.py               # Historical stress scenarios
│   ├── costs/
│   │   └── transaction_costs.py    # Slippage + commission model
│   └── backtest/
│       └── engine.py               # 8-step vectorized backtest pipeline
│
├── config/
│   ├── settings.py                 # AppSettings (Pydantic, env-overridable)
│   └── universes.yaml              # 27 ETFs, 4 asset classes, 3 presets
│
├── data/
│   └── cache/                      # Parquet price cache (auto-populated)
│
├── start_backend.py                # Backend launcher (sets sys.path)
├── docker-compose.yml              # Docker deployment
├── .github/workflows/ci.yml        # CI/CD pipeline
├── generate_thesis.py              # PDF thesis generator (ReportLab)
└── TSMOM_Thesis_Documentation.pdf  # 20-page academic documentation
```

Asset Universe (27 ETFs)

Asset Class	Ticker	Name	Sector
Equities	SPY	S&P 500	Broad Market
	QQQ	Nasdaq 100	Technology
	IWM	Russell 2000	Small Cap
	XLF	Financials Select	Financials
	XLE	Energy Select	Energy
	XLV	Healthcare Select	Healthcare
	XLK	Technology Select	Technology
	XLI	Industrials Select	Industrials
	XLP	Consumer Staples	Staples
	XLY	Consumer Discretionary	Discretionary
	XLU	Utilities Select	Utilities
	XLRE	Real Estate Select	Real Estate
Commodities	GLD	Gold	Precious Metals

Asset Class	Ticker	Name	Sector
	SLV	Silver	Precious Metals
	USO	Crude Oil	Energy
	UNG	Natural Gas	Energy
	DBA	Agriculture	Agriculture
Fixed Income	TLT	20+ Year Treasury	Long Duration
	IEF	7-10 Year Treasury	Mid Duration
	SHY	1-3 Year Treasury	Short Duration
	LQD	Investment Grade Corp	Credit
	HYG	High Yield Corp	High Yield
FX	UUP	US Dollar Index	Dollar
	FXE	Euro	Major
	FXY	Japanese Yen	Major
	FXB	British Pound	Major
	FXA	Australian Dollar	Commodity FX

Key Features

Signal Generation Pipeline

```
Raw Prices → Returns → Multi-Horizon Momentum → Regime Scaling → Smoothing → Z-
Score → Cap → Final Signal
                        (21, 63, 126, 252d)      (HMM 2-state)   (EMA)
(±2σ)    (±2.0)
```

- **Multi-horizon blending**: Combines 1-month, 3-month, 6-month, and 12-month momentum
- **Regime-aware**: HMM detects Trending vs Mean-Reverting regimes and scales signals accordingly
- **Robust processing**: Smoothing removes noise, z-score normalizes, cap prevents extreme bets

Volatility Estimators

Estimator	Type	Best For
EWMA	Exponentially Weighted	Fast-reacting, recent moves weighted more
Yang-Zhang	OHLC-based	Most efficient use of OHLC data
Garman-Klass	OHLC-based	Robust to drift, uses full price range

Estimator	Type	Best For
Rolling StdDev	Simple	Baseline comparison

Portfolio Construction (MOP Framework)

- **Volatility Targeting**: Each position sized to contribute equally to portfolio risk
- **Position Limits**: Max 10% per position (configurable)
- **Gross Exposure Cap**: Max 2.0x leverage (configurable)
- **Rebalancing**: Monthly (or daily/weekly), with turnover tracking
- **Drawdown Protection**: Automatically scales down exposure when drawdown exceeds threshold

Risk Management

Metric	Description
Sharpe Ratio	Risk-adjusted return (excess return / volatility)
Sortino Ratio	Downside risk-adjusted return
Calmar Ratio	Return / max drawdown
VaR (95%, 99%)	Value-at-Risk at confidence levels
CVaR (95%, 99%)	Expected shortfall beyond VaR
Max Drawdown	Worst peak-to-trough decline
Hit Rate	Percentage of profitable trading days
Skewness	Return distribution asymmetry
Kurtosis	Tail fatness of returns
Information Ratio	Active return / tracking error vs benchmark
Beta / Alpha	Market sensitivity and excess return

Historical Stress Tests

The system tests your portfolio against major market events:

- **2008 Global Financial Crisis**
- **2010 Flash Crash**
- **2011 US Debt Downgrade**
- **2013 Taper Tantrum**
- **2015 China Devaluation**
- **2018 Volmageddon**
- **2020 COVID-19 Crash**
- **2022 Rate Hike Cycle**

Configuration

All parameters are configurable via environment variables (prefix: `TSMOM_`) or `config/settings.py`:

Parameter	Default	Description
`vol_target`	0.10	Annual volatility target (10%)
`momentum_lookbacks`	[21, 63, 126, 252]	Signal lookback windows (days)
`rebalance_frequency`	monthly	Rebalancing schedule
`drawdown_threshold`	0.10	Drawdown level to trigger protection
`drawdown_scaling_floor`	0.25	Minimum exposure during drawdown
`max_position_weight`	0.10	Maximum weight per asset
`max_gross_exposure`	2.0	Maximum gross leverage
`default_slippage_bps`	5.0	Slippage per trade (basis points)
`commission_bps`	1.0	Commission per trade (basis points)
`signal_cap`	2.0	Maximum absolute signal value

Strategy Presets

Preset	Vol Target	Lookbacks	Rebalance	DD Threshold
Conservative	8%	[126, 252]	Monthly	8%
Balanced	10%	[21, 63, 126, 252]	Monthly	10%
Aggressive	15%	[21, 63]	Weekly	15%

How the Frontend Connects to the Backend

```
Browser (localhost:3000)
    │
    ▼
Next.js Frontend
    │  fetch("http://localhost:8000/api/run-backtest", { method: "POST", body:
JSON.stringify(params) })
    │
    ▼
FastAPI Backend (localhost:8000)
    │  CORS: allow_origins=["*"]
    │
    ▼
Service Layer (engine.py)
    │  1. Check in-memory cache (10-min TTL)
    │  2. If miss → load prices from Parquet cache
    │  3. Run quant engine computation
    │  4. Cache result, return JSON
```

```
    │
    ▼
Quant Engine (src/)
    │  BacktestEngine.run(prices) → BacktestResult
    │
    ▼
Data Layer (data/cache/)
    └── Parquet files with 27 ETFs × 20+ years of daily OHLCV
```

The connection is configured in `frontend/.env.local`:

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api
```

Performance

Metric	Value
Cold backtest (10 assets, 20yr)	~10 seconds
Cached backtest response	~0.4 seconds
Frontend build size	~163 KB first load JS
Data cache size	~50 MB (27 ETFs, 2005-present)
API endpoints	9
Frontend pages	8
Computed metrics	30+
Asset universe	27 ETFs across 4 classes

Deployment

Docker

```
docker-compose up --build
```

This starts:

- **Backend** on port 8000
- **Frontend** on port 3000

Production

```
# Backend
cd backend && uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4

# Frontend
cd frontend && npm run build && npm start
```

Troubleshooting

Problem	Solution
Port 8000 already in use	`taskkill /F /IM python.exe` (Windows) or `kill -9 $(lsof -ti:8000)` (Mac/Linux)
Port 3000 already in use	`taskkill /F /IM node.exe` (Windows) or `kill -9 $(lsof -ti:3000)` (Mac/Linux)
`ModuleNotFoundError` (Python)	Run `pip install -r backend/requirements.txt` from project root
`Module not found` (Node)	Run `cd frontend && npm install`
Backend 500 error on backtest	Check that `data/cache/` has Parquet files. If empty, the system will try to download via yfinance (needs internet)
CORS error in browser	Backend CORS is set to `["*"]` — should work. Clear browser cache and retry
Slow first load	Normal. First call computes 20 years of backtest data. Subsequent calls hit the 10-minute cache
Frontend shows "Failed to load"	Make sure the backend is running first (`python start_backend.py`)

Technology Stack

Backend

Technology	Purpose
FastAPI 0.115	Async REST API framework
Uvicorn	ASGI server with hot reload
Pandas / NumPy / SciPy	Numerical computation
scikit-learn	Statistical utilities
hmmlearn	Hidden Markov Model regime detection

Technology	Purpose
yfinance	Market data provider
SQLAlchemy + aiosqlite	Async database (future use)
Pydantic v2	Settings validation & serialization
PyArrow	Parquet file I/O

Frontend

Technology	Purpose
Next.js 15	React framework with App Router
React 19	UI component library
TypeScript 5.7	Type-safe JavaScript
Plotly.js	Interactive financial charts
TanStack React Query	Server state management & caching
Framer Motion	Smooth animations & transitions
Tailwind CSS 3.4	Utility-first styling
Radix UI	Accessible headless components
next-themes	Dark/light mode
KaTeX	LaTeX math rendering
Lucide React	Icon library

UI Design

The frontend features a **futuristic glassmorphism** design language:

- **Glassmorphic cards** — Translucent backgrounds with blur and saturation
- **Animated gradient mesh** — Floating ambient orbs in the background
- **3D card hover** — Cards lift and tilt on hover with perspective transform
- **Glow effects** — Green glow for profit, red for loss, purple for primary
- **Gradient text** — Headings use purple-to-blue gradient fills
- **Animated borders** — Key cards have gradient border animations
- **Shimmer loading** — Skeleton loaders with sweeping highlight animation
- **Noise texture** — Subtle grain overlay for depth
- **Spring physics** — KPI cards animate in with spring-based motion

---

Academic Foundation

This project implements the strategy described in:

> **Moskowitz, T. J., Ooi, Y. H., & Pedersen, L. H. (2012)**. "Time Series Momentum."
> *Journal of Financial Economics, 104(2), 228–250.*

Key concepts implemented:

- Time-series momentum signal construction
- Volatility scaling for position sizing
- Multi-asset diversification across asset classes
- Transaction cost modeling
- Drawdown-aware portfolio management

A complete 20-page academic thesis is included: `TSMOM_Thesis_Documentation.pdf`

License

This project is for educational and research purposes. Not financial advice.

Built with the MOP Framework | Quantitative Finance Research Platform